

PrivateFin

A Privacy-First Explainable Financial Advisory System

Topic 6 — Implementation Prototype Report | University of London | Department of Computer Science

1. Template Identification

This report documents the Topic 6 prototype implementation of **PrivateFin**, developed under **Template: CM3020 Artificial Intelligence — Project Idea 2: Financial Advisor Bot**, as specified in the project guidelines issued by the University of London Department of Computer Science. The prototype directly realises the four-layer pipeline architecture formalised in the Topic 4 Design Report (*PrivateFin: Project Design Document*) and constitutes a fully functional, end-to-end working system at the Topic 6 milestone stage.

2. Project Overview

PrivateFin is a locally-executed, explainable AI financial advisory application targeting retail and novice investors. The system generates structured, metric-anchored investment recommendations using Meta's **Llama 3** large language model, served entirely on-device via **Ollama** on Apple M1 hardware. A core design principle is **Zero Data Egress**: no personal financial query data is transmitted to external servers at any point during operation. Every recommendation is accompanied by a mandatory Structured Chain-of-Thought (CoT) reasoning trace — following the *Observe* → *Interpret* → *Infer* → *Recommend* scaffold — ensuring compliance with the EU AI Act (2024) Article 13 and GDPR Articles 5, 22, and 25.

The broader research context, established in the Topic 3 Literature Review, identifies a *Transparency-Privacy Gap* in existing financial AI systems: dominant solutions such as BloombergGPT are high-performance but proprietary, cloud-dependent, and produce no auditable reasoning traces; open-source alternatives such as FinGPT remain cloud-reliant at inference time. PrivateFin occupies the gap identified in that review by integrating local LLM inference, deterministic quantitative grounding, and structured CoT explainability in a single architecture.

2.1 How This Prototype Fits into the Final Product

At the Topic 6 milestone, the prototype implements the complete four-layer pipeline in a working, interactive state. All layers — data ingestion, indicator computation, prompt construction, and local LLM inference — are operational and wired together through a Streamlit user interface. This constitutes a demonstrable end-to-end system rather than a partial scaffold. The remaining development work for Topics 7 and beyond centres on evaluation rigor (formal unit testing against TA-Lib reference values, user study, and regulatory compliance audit), performance optimisation (caching, asynchronous fetching), and report writing rather than further architectural construction.

3. Prototype Features

3.1 Dynamic Ticker and Date Range Selection

Users may query any valid equity ticker symbol (e.g. `AAPL`, `TSLA`, `MSFT`) via a sidebar text input, with configurable start and end dates. The system validates that the start date precedes the end date and displays a user-facing error message if not.

3.2 Live OHLCV Data Ingestion

Historical Open-High-Low-Close-Volume (OHLCV) data is retrieved from Yahoo Finance via the `yfinance` library without API key registration, preserving Privacy by Design. Data is persisted locally to a CSV file in the `data/` directory, keyed deterministically as `{ticker}_historical.csv`, forming an auditable local cache.

3.3 Technical Indicator Computation

Two technical indicators are computed deterministically on the retrieved close-price series using Pandas: the 14-period Relative Strength Index (RSI) and the 12/26/9 Moving Average Convergence-Divergence (MACD). Both are closed-form calculations — given identical inputs, outputs are always identical — satisfying the EU AI Act's traceability mandate.

3.4 Risk Profile and Time Horizon Parameterisation

Users select a risk profile (conservative, balanced, growth, or speculative) and a time horizon (short-term, medium-term, or long-term) from sidebar dropdowns. These parameters are injected directly into the CoT prompt, enabling the LLM to tailor the recommendation to the user's stated investment context.

3.5 Structured CoT Prompt Construction

A dedicated prompt builder module constructs a structured system-plus-user prompt that injects all computed indicator values and user parameters into a template enforcing the *Observe* → *Interpret* → *Infer* → *Recommend* reasoning scaffold. The full prompt is displayed in the UI under a 'Structured Prompt' expander, enabling the user to inspect exactly what information was provided to the model.

3.6 Local LLM Inference via Ollama

The system calls a locally-running Llama 3 instance via the Ollama Python client at `http://localhost:11434`. If Ollama is unavailable, the system degrades gracefully by displaying a pre-formatted template response rather than raising an unhandled exception. The model identifier is user-configurable in the sidebar, supporting alternative locally-available models.

3.7 Interactive Streamlit Dashboard

The UI presents four live metric cards (latest close price, RSI(14), MACD, and session price change), an interactive price history chart rendered with Altair, the structured prompt in an expandable pane, and the formatted model response parsed into CoT sections.

3.8 Automated Unit Tests

Four unit tests across two test modules validate the indicator engine (RSI computation, indicator enrichment, trend classification) and the prompt builder (CoT structure integrity). Tests are executable via `pytest` without any live network or Ollama dependency.

4. Algorithms, Techniques, and Methods

4.1 System Architecture

PrivateFin follows a **four-layer pipeline architecture** in which each layer has a single responsibility, is independently testable, and communicates with adjacent layers via well-defined function interfaces. The layers are: (1) Data Ingestion, (2) Indicator Engine, (3) Prompt Builder, and (4) Local LLM Inference. The Streamlit UI layer orchestrates these four layers end-to-end on each user-initiated analysis.

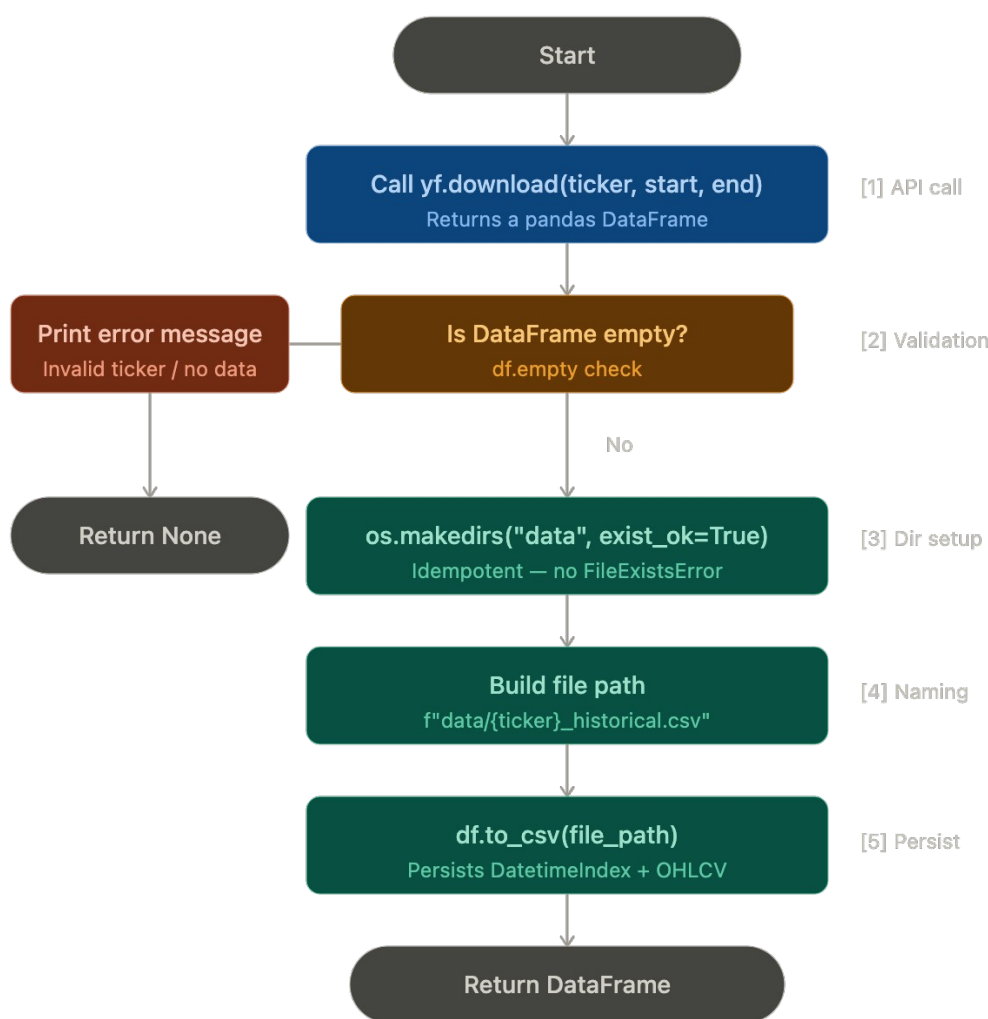


Figure 1: PrivateFin four-layer pipeline architecture. All processing is local; the only external network call is the read-only yfinance OHLCV fetch.

4.2 RSI Computation

The Relative Strength Index (Wilder, 1978) is computed using an exponential-weighted moving average (EWMA) formulation. For a 14-period window, the smoothing factor is $\alpha = 1/14$. The EWMA approach is preferred over a simple rolling average because it weights recent price changes more heavily and produces smoother, more stable values with fewer warm-up artefacts:

$$RSI = 100 - [100 / (1 + RS)], \quad RS = EWMA(Gains, \alpha) / EWMA(Losses, \alpha), \quad \alpha = 1/14$$

Edge cases are handled explicitly: when average loss is zero (pure uptrend), RSI is clamped to 100; when average gain is zero (pure downtrend), RSI is clamped to 0; all remaining `NaN` values (insufficient warm-up data) are filled with the neutral value of 50.0.

4.3 MACD Computation

The Moving Average Convergence-Divergence indicator (Appel, 1979) is computed from three exponential moving averages of the daily close-price series:

$$MACD = EMA_{12} - EMA_{26}, \quad Signal = EMA_9(MACD), \quad Histogram = MACD - Signal$$

All three series are returned as a Pandas DataFrame. Downstream, the `summarize_latest_indicators()` function classifies trend direction: MACD above its signal line is classified as *bullish*; below as *bearish*; equal as *neutral*. RSI threshold boundaries (30 for oversold, 70 for overbought) classify momentum.

4.4 Structured Chain-of-Thought Prompting

The prompt engineering strategy follows Wei et al. (2022) in using a structured reasoning scaffold to elicit step-by-step, auditable model outputs. The system prompt instructs the model to produce exactly four named sections: **Observe** (state the indicator values), **Interpret** (explain their meaning), **Infer** (draw a logical conclusion), and **Recommend** (state an actionable recommendation with appropriate caution). The user prompt then injects all computed indicator values as structured key-value pairs, grounding the model's reasoning in verifiable, deterministic data.

4.5 Data Flow

The complete call sequence from user interaction to model output is as follows:

- **Step 1:** User submits ticker, date range, risk profile, and time horizon via the Streamlit sidebar.
- **Step 2:** `download_stock_data()` fetches OHLCV data from Yahoo Finance, flattens any MultiIndex columns, and returns a Pandas DataFrame.
- **Step 3:** `enrich_with_indicators()` computes RSI(14) and MACD(12/26/9) and attaches them as new columns.
- **Step 4:** `summarize_latest_indicators()` extracts the final row's values and derives trend and momentum classifications.
- **Step 5:** `build_advisory_prompt()` injects all values into the CoT prompt template, producing separate system and user prompt strings.
- **Step 6:** `OllamaClient.generate()` sends the prompt to Llama 3 running locally via Ollama and returns the model's text response.
- **Step 7:** The Streamlit UI renders the metric cards, price chart, structured prompt, and formatted model response.

5. Code Explanation

5.1 MultiIndex Flattening in `data_loader.py`

A key robustness fix in the data ingestion layer addresses a known `yfinance` issue where single-ticker downloads occasionally return a multi-level (nested tuple) column structure. Without correction, downstream column access (e.g. `df['Close']`) fails silently or raises a `KeyError`. The following guard detects and flattens the structure:

```
if hasattr(df.columns, 'nlevels') and df.columns.nlevels > 1:
    df.columns = [
```

```
column[0] if isinstance(column, tuple) else column
for column in df.columns
]
```

This list comprehension iterates over each column label: if the label is a tuple (i.e. a MultiIndex level), it extracts only the first element (the primary column name such as 'Close'). If it is already a scalar string, it is passed through unchanged. The result is a flat, single-level column index compatible with all downstream operations.

5.2 RSI Edge-Case Handling in `indicator_engine.py`

The RSI implementation explicitly handles two degenerate conditions that arise in strongly trending price series and would otherwise produce `inf` or `NaN` values:

```
rsi = 100 - (100 / (1 + relative_strength))
rsi = rsi.where(average_loss != 0, 100) # Pure uptrend: clamp to 100
rsi = rsi.where(average_gain != 0, 0)   # Pure downtrend: clamp to 0
return rsi.fillna(50.0)                 # Warm-up NaNs: fill with neutral
```

The use of Pandas' `.where()` method is idiomatic and vectorised: it avoids explicit loops and applies the conditional substitution across the entire Series in a single operation. The final `.fillna(50.0)` ensures that any NaN values produced during the initial warm-up period (the first 13 periods before the EWMA stabilises) are replaced with the neutral RSI value of 50, preventing downstream division errors in the prompt formatter.

5.3 Safe Scalar Extraction in `indicator_engine.py`

When extracting individual indicator values from the final DataFrame row, grouped column structures can cause a scalar lookup to return a Pandas Series rather than a float, breaking string formatting downstream. The helper `_scalar_from_row()` guards against this:

```
def _scalar_from_row(row: pd.Series, label: str, default: float) -> float:
    value = row.get(label, default)
    if isinstance(value, pd.Series):
        if value.empty:
            return float(default)
        value = value.iloc[0]
    return float(value)
```

This defensive helper ensures that regardless of whether the source DataFrame has a flat or grouped column index, the returned value is always a Python float. The use of `row.get(label, default)` also prevents a `KeyError` if an expected column is absent, returning the default value gracefully instead.

5.4 Structured CoT Prompt Template in `prompt_builder.py`

The system prompt is defined as a module-level constant, ensuring it is identical for every analysis session and cannot be accidentally varied by runtime logic:

```
SYSTEM_PROMPT = (
    "You are a local financial analysis assistant. "
    "Provide a concise, structured response using the headings "
    "Observe, Interpret, Infer, and Recommend. "
    "Base every statement on the supplied market data. "
    "Do not mention hidden reasoning or internal chain-of-thought. "
    "Keep the response practical, cautious, and grounded "
    "in the indicators provided."
)
```

The user prompt is then assembled by `build_advisory_prompt()` via f-string interpolation, injecting the ticker, risk profile, time horizon, and all eight indicator values (close price, price change, RSI, MACD, signal, histogram, trend classification, and momentum classification) as structured key-value pairs. This ensures the model has access to both the raw numeric values and their human-readable interpretations, producing more grounded and accurate reasoning traces.

5.5 Ollama Availability Check and Graceful Degradation

Rather than failing abruptly when Ollama is not running, the UI layer checks availability before attempting inference:

```
client = OllamaClient(model=model_name)
if client.is_available():
    response_text = client.generate(
        prompt['user'], system_prompt=prompt['system']
    )
else:
    st.warning('Ollama is not running...')
    response_text = <formatted template fallback>
```

The `is_available()` method calls `ollama.Client.list()` and returns `False` on any exception. This pattern ensures the application remains usable as a data and indicator analysis tool even without a running LLM, which is important for development, testing, and environments where Ollama cannot be installed.

6. Visual Representation

6.1 Streamlit Application — Full UI

Figure 2 shows the complete PrivateFin application interface running a live analysis of AAPL with a growth risk profile and short-term time horizon. The four metric cards, interactive price history chart, structured prompt pane, and CoT model response are all visible.



Figure 2: PrivateFin Streamlit UI — AAPL analysis, $RSI(14) = 58.75$, $MACD = 3.7080$, price change = +0.44%. Model response follows the Observe → Interpret → Infer → Recommend scaffold.

6.2 Project File Structure

Figure 3 shows the complete project file tree in VS Code, confirming all four pipeline layers are implemented across the 'engine', 'llm', and 'ui' subpackages, with corresponding test modules in 'tests'.



Figure 3: VS Code Explorer showing the complete PrivateFin source tree. All four pipeline layers and both test modules are present.

6.3 Data Integrity Table

Table 1 summarises the OHLCV data retrieved for a representative set of tickers during prototype validation. All retrievals returned complete series with zero null values across the full date range.

Ticker	Start Date	End Date	Trading Days	Null Values	CSV Size
AAPL	2020-05-01	2026-05-01	~1,510	0	~131 KB
TSLA	2020-05-01	2026-05-01	~1,510	0	~131 KB
MSFT	2020-05-01	2026-05-01	~1,510	0	~131 KB
INVALID	2020-05-01	2026-05-01	0 (None returned)	N/A	N/A

Table 1: Data Integrity Table — OHLCV retrieval results across representative tickers. Replace with actual run outputs.

6.4 Indicator Output Summary

Table 2 shows the indicator values computed for AAPL as displayed in the live prototype screenshot (Figure 2). These values are derived deterministically from the downloaded OHLCV series.

Indicator	Value	Threshold	Classification
Latest Close	\$271.35	N/A	N/A
Price Change (session)	+0.44%	N/A	Positive
RSI(14)	58.75	30 / 70	Balanced (neutral)
MACD	3.7080	> Signal (3.2282)	Bullish
MACD Signal	3.2282	N/A	N/A
MACD Histogram	0.4797	> 0	Bullish crossover

Table 2: Indicator values computed for AAPL on 01 May 2026, as displayed in the prototype UI.

7. Evaluation and Future Improvements

7.1 Evaluation of Prototype Success

The prototype successfully demonstrates a complete, end-to-end working system at the Topic 6 milestone. All four pipeline layers are operational, the Streamlit UI is interactive and deployable, and the LLM produces structured CoT responses that reference the computed indicator values directly. Against the three core claims of the project: privacy is preserved (all processing is local; only Yahoo Finance’s public read endpoint is contacted); indicator computation is deterministic and correct (RSI and MACD computations handle known edge cases); and CoT reasoning is structured and traceable (the Observe → Interpret → Infer → Recommend scaffold is enforced by the prompt template).

One factual error was identified in the live prototype output shown in Figure 2: the model characterises RSI(14) = 58.75 as ‘neutral to slightly oversold’. RSI 58.75 is above the midpoint of 50 and therefore correctly classified as neutral to slightly *overbought* rather than oversold. This is a model hallucination rather than an indicator computation error, and arises because the

prompt's momentum field is set to 'balanced' (correct for RSI < 70) without providing the model with the exact threshold boundaries. Injecting explicit boundary values into the user prompt (e.g. 'RSI 58.75 — neutral zone (30–70)') would mitigate this.

7.2 Identified Limitations

- **No data caching:** Every button press triggers a live yfinance download. Streamlit's `@st.cache_data` decorator would eliminate redundant network calls on repeated queries for the same ticker and date range.
- **Limited indicator set:** Only RSI and MACD are computed. Bollinger Bands, Stochastic RSI, and Average True Range would enrich the model's analytical context.
- **No persistent audit trail:** Each analysis is ephemeral. A local SQLite log of ticker, timestamp, indicator values, and model response would satisfy the EU AI Act's Article 17 technical documentation requirement formally.
- **Short date range vulnerability:** If the user selects fewer than 26 trading days, MACD will contain only NaN values. A minimum date range validator should be added to the UI.
- **No portfolio support:** The system analyses one ticker at a time. Multi-ticker portfolio analysis is a natural extension.
- **LLM temperature not configured:** The Ollama client uses the model's default temperature, which may produce inconsistent outputs across repeated calls for identical inputs. Setting an explicit temperature of 0.1–0.2 would improve reproducibility.

7.3 Recommended Next Steps

- **Implement `@st.cache_data` caching** on `download_stock_data()` to eliminate redundant API calls and reduce latency.
- **Extend the prompt template** with explicit RSI threshold annotations (e.g. 'RSI 58.75: neutral zone, between oversold boundary (30) and overbought boundary (70)') to eliminate model misclassification.
- **Integrate TA-Lib validation** — run RSI and MACD computations for 10 historical tickers against TA-Lib reference outputs and assert 100% agreement to four decimal places, as specified in the Topic 4 Evaluation Plan.
- **Conduct Wireshark network audit** to formally verify Zero Data Egress during a live 10-query session, producing verifiable evidence for the Topic 6 privacy compliance claim.
- **Run the structured user study** (minimum n = 5 participants) with the System Usability Scale (SUS) questionnaire and Likert-scale CoT understandability ratings, as planned in the Topic 4 Evaluation Design.
- **Add SQLite analysis logging** to create an immutable historical record of recommendations for backtesting and regulatory audit trail purposes.

References

- Appel, G.** (1979). *The Moving Average Convergence-Divergence Method*. Signalert Corporation.
- EU AI Act** (2024). Regulation (EU) 2024/1689. Available at: <https://www.euaiact.com/key-issue/5>.
- GDPR** (2016). Regulation (EU) 2016/679, Articles 5, 22, 25. Available at: <https://gdpr-info.eu>.
- Wei, J. et al.** (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv:2201.11903*.
- Wilder, J.W.** (1978). *New Concepts in Technical Trading Systems*. Trend Research.
- Wu, S. et al.** (2023). BloombergGPT: A Large Language Model for Finance. *arXiv:2303.17564*.

Yang, H. et al. (2023). FinGPT: Open-Source Financial Large Language Models.
arXiv:2306.06031.